

Lección 3. Evolución del aprendizaje profundo y otras técnicas modernas de ML

El aprendizaje profundo es una rama del aprendizaje automático que se basa en redes neuronales con múltiples capas ocultas. Estas redes profundas son capaces de aprender representaciones complejas de los datos de forma jerárquica, lo que las hace especialmente útiles en tareas como reconocimiento de imágenes, procesamiento de lenguaje o análisis de series temporales.

A lo largo de esta unidad hemos explorado las arquitecturas más representativas del aprendizaje profundo, como las redes multicapa (MLP), las redes recurrentes (RNN y LSTM), los autoencoders, las redes convolucionales (CNN) y las redes generativas adversarias (GAN). Todas ellas forman parte de lo que se considera deep learning, siempre que cuenten con una cierta profundidad y estén compuestas por capas que se entrenan conjuntamente.

En esta lección final nos centraremos en dos enfoques muy utilizados en la práctica, aunque no siempre se consideran deep learning en sentido estricto. El primero es el *transfer learning*, que sí lo es cuando se apoya en redes profundas ya entrenadas, y que permite aprovechar el conocimiento de modelos existentes para resolver nuevos problemas con menos datos y menos tiempo. El segundo son las *metaheurísticas*, técnicas de optimización inspiradas en procesos naturales o estrategias inteligentes, que se utilizan para encontrar soluciones en problemas complejos donde los métodos tradicionales no funcionan bien. Aunque estas técnicas no pertenecen al aprendizaje profundo, pueden integrarse en flujos de trabajo de machine learning para mejorar resultados.

Existen otras tendencias modernas que no abordaremos aquí, como las redes basadas en atención, los transformadores o los modelos de difusión. Son herramientas muy potentes en tareas avanzadas, pero requieren un nivel de profundidad adicional que se verá en etapas más avanzadas del aprendizaje.

Transfer Learning

En el contexto del aprendizaje profundo, una técnica especialmente útil y extendida es el aprendizaje por transferencia o *transfer learning*. Esta estrategia consiste en aprovechar un modelo previamente entrenado en una tarea (normalmente con grandes volúmenes de datos y mucha capacidad computacional) y adaptarlo a una nueva tarea que suele disponer de menos datos o requerir un entrenamiento más rápido.

El objetivo no es empezar desde cero, sino reutilizar el conocimiento que el modelo ya ha adquirido, como si tuviera una base previa sobre cómo interpretar ciertos datos. Esto permite ahorrar tiempo, reducir el uso de recursos y, en muchos casos, mejorar el rendimiento en tareas específicas.

Aunque esta técnica es especialmente útil cuando no se dispone de grandes conjuntos de datos, también se utiliza cuando sí los hay, ya que permite acelerar el proceso de entrenamiento o evitar sobreajuste. Es una estrategia muy usada en visión por computador, procesamiento de lenguaje natural y muchos otros ámbitos actuales.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



En este apartado veremos en qué consiste exactamente el *transfer learning*, cuándo es recomendable usarlo, qué ventajas ofrece y cómo se aplica en la práctica, centrándonos en el uso de redes convolucionales previamente entrenadas.

Qué es y cómo funciona

Cuando se aplica *transfer learning*, no se entrena un modelo desde cero, sino que se parte de uno ya entrenado sobre una tarea distinta. Para adaptarlo a nuestro nuevo problema, se siguen generalmente dos fases: primero se congelan partes del modelo y luego, si es necesario, se ajustan mediante *fine-tuning*. Cada fase cumple una función distinta y tiene ventajas según el tipo de datos y el objetivo.

Fase de congelación

La congelación de capas es la primera fase típica en un proceso de *transfer learning*. Consiste en mantener los pesos ya aprendidos por el modelo preentrenado sin modificarlos durante el nuevo entrenamiento. Esto se hace para conservar el conocimiento general adquirido en tareas anteriores, especialmente si la tarea nueva es similar a la original o si se dispone de pocos datos.

Al congelar una capa, se evita que sus pesos se actualicen. En la práctica, esto se traduce en que el optimizador no modifica esas capas durante el entrenamiento, lo cual reduce el tiempo de entrenamiento y el riesgo de sobreajuste.

Dependiendo del objetivo y de la cantidad de datos disponibles, existen varias estrategias:

1. **Congelar todas las capas del modelo original**

Esta es la opción más segura cuando se trabaja con un conjunto de datos pequeño y muy parecido al que se usó para entrenar el modelo base. Por ejemplo, si se parte de una CNN entrenada con imágenes de perros y gatos, y el nuevo conjunto también contiene animales domésticos, muchas características visuales aprendidas ya son válidas. En este caso, solo se añade una nueva capa final adaptada al número de clases deseado, y se entrena exclusivamente esa parte. Es útil para tareas de clasificación rápida con pocos recursos.

2. **Congelar todas menos la última capa del modelo original**

A veces interesa mantener la mayoría del conocimiento previo, pero permitir que la última capa del modelo original se ajuste ligeramente a los nuevos datos. Esto se hace cuando la tarea es similar, pero no idéntica. Por ejemplo, pasar de una clasificación de 10 tipos de flores a una clasificación de 5 especies distintas de árboles puede beneficiarse de esta pequeña adaptación.

3. **Congelar solo las capas iniciales y descongelar las últimas**

En casos donde el nuevo conjunto de datos es más amplio o tiene diferencias importantes respecto al conjunto original, se pueden congelar únicamente las primeras capas del modelo. Estas capas suelen aprender patrones muy genéricos (líneas, bordes, texturas), que son útiles para muchas tareas. Las últimas capas, más específicas, sí se descongelan y se entrenan con los nuevos datos para adaptarse a las particularidades del nuevo problema.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



4. No congelar ninguna capa

Esta opción se utiliza cuando el conjunto nuevo de datos es grande y bastante diferente, y se necesita adaptar todo el modelo. En este caso, se entrena todo desde el principio, pero partiendo de pesos ya preentrenados en lugar de valores aleatorios. Aun así, se recomienda usar una tasa de aprendizaje muy baja para no dañar el conocimiento útil ya aprendido.

Ejemplo

Supongamos que partimos de una red CNN preentrenada en ImageNet y queremos usarla para clasificar imágenes médicas (por ejemplo, radiografías). Como estas imágenes son muy diferentes de las fotos naturales de ImageNet, no basta con cambiar la última capa. En este caso, podríamos:

- Congelar las primeras capas convolucionales (que aprenden bordes y patrones básicos)
- Dejar entrenables las últimas capas convolucionales y las nuevas capas densas que añadimos
- Entrenar primero solo las nuevas capas
- Después, si el rendimiento no es suficiente, descongelar progresivamente más capas y afinar el modelo (fase de *fine-tuning*)

Esta estrategia permite que el modelo conserve lo aprendido, pero también se adapte progresivamente al nuevo dominio.

Fase de fine-tuning

Una vez congeladas las capas del modelo base y añadidas las nuevas capas para adaptar la red al problema específico, el siguiente paso consiste en ajustar con precisión los pesos de todo el modelo, o parte de él, a los datos concretos. A este proceso se le llama fine-tuning (ajuste fino). No se trata de entrenar la red desde cero, sino de refinar los conocimientos previamente aprendidos para adaptarlos a la nueva tarea.

El fine-tuning suele realizarse en una segunda etapa del entrenamiento. En la primera, se entrenan solo las capas nuevas añadidas (por ejemplo, una capa densa final), mientras que el resto permanece congelado. En esta segunda etapa, se descongelan algunas capas del modelo base —normalmente las más profundas— para permitir que se ajusten a las características del nuevo conjunto de datos.

Cuántas capas se deben descongelar dependerá de la similitud entre el nuevo problema y el original. Si son muy parecidos (por ejemplo, imágenes médicas tomadas con técnicas similares), con ajustar solo las últimas capas suele ser suficiente. Si son distintos (por ejemplo, pasar de imágenes de animales a imágenes de satélite), puede convenir descongelar más capas o incluso todo el modelo.

Durante esta fase es importante usar una tasa de aprendizaje baja para evitar sobrescribir el conocimiento ya adquirido. De lo contrario, el modelo podría olvidar lo aprendido inicialmente (un fenómeno conocido como catástrofe del olvido) y perder la ventaja de haber sido preentrenado.

Un flujo típico sería el siguiente:

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



- entrenar solo la nueva capa de salida con las capas base congeladas
- evaluar resultados
- descongelar algunas capas profundas del modelo base
- continuar entrenando todo el modelo con tasa de aprendizaje baja

Este proceso permite aprovechar la capacidad del modelo base para extraer características generales, pero adaptándolas con precisión al nuevo contexto.

La diferencia clave entre la fase de congelación y la de fine-tuning está en qué partes del modelo se entrenan en cada momento. En la fase de congelación, se entrena solo la nueva parte final añadida (normalmente una o varias capas Dense adaptadas al nuevo problema), mientras que el resto del modelo se mantiene fijo para aprovechar el conocimiento aprendido previamente. En cambio, en la fase de fine-tuning, se permite ajustar también algunas capas del modelo original, afinando sus pesos con los nuevos datos. Por ejemplo, si se parte de un modelo entrenado con imágenes generales y se quiere clasificar tipos de flores, primero se congela todo salvo la última capa específica para flores, y más adelante, cuando esa parte ya funciona bien, se descongelan algunas capas del modelo base para afinar la red con ese nuevo dominio.

Casos comunes y ventajas

El *transfer learning* es especialmente útil en situaciones donde entrenar una red neuronal desde cero no es práctico. Algunos de los casos más habituales son aquellos en los que no se dispone de suficientes datos etiquetados, o cuando se quiere aprovechar el conocimiento aprendido por una red muy compleja entrenada previamente con grandes volúmenes de datos. También se usa cuando el problema que se quiere resolver es similar al del modelo original, como por ejemplo utilizar una red entrenada con imágenes de objetos cotidianos para clasificar imágenes médicas.

Una de las ventajas más claras del *transfer learning* es que permite reducir drásticamente el tiempo de entrenamiento, ya que no es necesario ajustar todos los parámetros desde cero. También es útil cuando no se dispone de grandes recursos computacionales, o cuando el conjunto de datos es limitado pero se busca una buena capacidad de generalización.

En resumen, esta técnica permite aprovechar modelos potentes ya entrenados para resolver problemas nuevos, adaptándolos a diferentes contextos de manera eficiente y con buenos resultados.

Transfer learning con CNN

Las redes convolucionales (CNN) son uno de los tipos de modelos donde más se ha extendido el uso del *transfer learning*. Esto se debe a que las primeras capas de una CNN aprenden a detectar patrones visuales muy generales, como bordes, texturas o formas simples, que son útiles en casi cualquier tarea relacionada con imágenes.

Por eso, muchos modelos de visión por computador, como VGG, ResNet o MobileNet, se entrenan previamente con bases de datos masivas como ImageNet y se publican como modelos preentrenados. Estos modelos pueden reutilizarse como punto de partida para tareas nuevas, incluso

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



si el conjunto de datos actual es pequeño o diferente. Basta con adaptar la parte final del modelo a la nueva tarea, normalmente sustituyendo las últimas capas densas por otras que se ajusten al número de clases deseado.

Además, al ser modelos ampliamente estudiados y optimizados, se dispone de pesos bien ajustados, lo que permite aprovechar al máximo las etapas de congelación y *fine-tuning*. Por ejemplo, en una tarea de clasificación de imágenes médicas, se puede usar una ResNet entrenada con ImageNet, conservar sus capas convolucionales y añadir nuevas capas densas específicas para clasificar radiografías o tomografías.

Aunque las CNN son las más habituales en transfer learning, no son las únicas. También se pueden aplicar técnicas similares con otros modelos, como redes recurrentes (RNN o LSTM) en tareas de texto o series temporales, o modelos tipo transformer en procesamiento de lenguaje natural. En estos casos, el proceso es el mismo: aprovechar una parte del modelo ya entrenado con grandes corpus y ajustarlo a un nuevo conjunto de datos con menos volumen o diferente enfoque.

Este enfoque combina el aprendizaje profundo con la eficiencia del *transfer learning*, obteniendo resultados sólidos sin necesidad de grandes recursos ni de entrenar modelos desde cero. La clave está en identificar qué parte del modelo es reutilizable y qué parte debe adaptarse para resolver el nuevo problema.

Ejemplo básico de Transfer Learning

```
# Cargar modelo base preentrenado
base_model = tf.keras.applications.MobileNetV2(input_shape=(128, 128, 3),
                                                include_top=False,
                                                weights='imagenet')

base_model.trainable = False # Fase de congelación

# Añadir nuevas capas para la tarea específica
model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(1, activation='sigmoid') # Clasificación binaria
])

# Compilar y entrenar solo la parte nueva
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])
model.fit(train_data, train_labels, epochs=5, validation_split=0.2)

# Fase de fine-tuning: descongelar últimas capas del modelo base
base_model.trainable = True
for layer in base_model.layers[:-20]: # Mantener congeladas las capas
    layer.trainable = False

# Recompilar con tasa de aprendizaje más baja
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
              loss='binary_crossentropy',
```

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



```
metrics=['accuracy'])

# Entrenamiento adicional con fine-tuning
model.fit(train_data, train_labels, epochs=5, validation_split=0.2)
```

Este código aplica transferencia de aprendizaje con una red convolucional preentrenada. Primero se utiliza MobileNetV2 como extractor de características, congelando sus capas para que no se modifiquen. A continuación, se incorpora una nueva parte final al modelo: un bloque con reducción de dimensiones, una capa densa y una salida sigmoide que aprende a clasificar según el nuevo conjunto de datos. En la segunda fase, se descongelan algunas capas profundas del modelo original para ajustar mejor los patrones internos a la nueva tarea. Aunque el modelo ya funciona tras el primer entrenamiento, este ajuste fino mejora el rendimiento final aprovechando el conocimiento previo de la red y adaptándolo al nuevo contexto.

Metaheurísticas aplicadas al ML

En muchos problemas de machine learning, no basta con elegir un buen modelo. También hay que tomar decisiones importantes como qué hiperparámetros usar o qué variables conservar. Cuando hay muchas combinaciones posibles y no existe una fórmula directa para encontrar la mejor, se utilizan las metaheurísticas.

Las metaheurísticas son técnicas de búsqueda avanzadas que permiten explorar conjuntos muy grandes de soluciones posibles (lo que llamamos *espacio de búsqueda*) sin necesidad de probarlas todas. A diferencia de métodos clásicos como la búsqueda exhaustiva o el descenso por gradiente, no siguen reglas fijas, sino que se apoyan en estrategias heurísticas (basadas en la experiencia o en reglas aproximadas que funcionan bien en la práctica).

Estas técnicas se usan para encontrar soluciones buenas en problemas complejos, incluso cuando no se conoce bien la forma de la función que queremos optimizar o hay múltiples soluciones válidas.

En este tema veremos cómo se aplican algunas de estas técnicas al aprendizaje automático. Aunque hay muchas (como búsqueda tabú o GRASP), nos centraremos en los algoritmos genéticos, por su facilidad de uso y su eficacia en tareas reales de optimización.

Qué son y para qué sirven los algoritmos genéticos

Los algoritmos genéticos son métodos de optimización inspirados en la evolución natural. En lugar de buscar una única solución directamente, trabajan con un conjunto de soluciones posibles (llamado población), que se van mejorando generación tras generación mediante mecanismos como la selección, el cruce y la mutación.

Cada solución, o individuo, representa una posible combinación de valores. En machine learning, por ejemplo, puede ser un conjunto de hiperparámetros (como la profundidad de un árbol, la tasa de aprendizaje o el número de neuronas de una red). Estas soluciones se evalúan con una función de puntuación (fitness), y las mejores se combinan y modifican para generar nuevas soluciones. Con el tiempo, el algoritmo converge hacia valores óptimos o cercanos al óptimo.

Se utilizan en ML especialmente para:

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



- Ajustar hiperparámetros sin necesidad de probar todas las combinaciones posibles
- Seleccionar variables relevantes entre muchas, mejorando el rendimiento y reduciendo el ruido
- Resolver problemas donde el espacio de soluciones es muy grande, no tiene estructura clara o está lleno de óptimos locales

Lo más útil de los algoritmos genéticos es que no necesitan derivadas ni fórmulas exactas, por lo que se adaptan bien a funciones complicadas, discontinuas o con ruido. Por eso son una alternativa potente a métodos clásicos como *grid search* o *random search*, especialmente cuando el número de combinaciones posibles es muy alto.

Preparación de los datos

Si el modelo necesita que los datos estén normalizados, escalados o codificados, este preprocesamiento debe hacerse antes de empezar, igual que en el resto de modelos vistos. El uso de un algoritmo genético no sustituye esa fase. Simplemente se encarga de buscar las mejores combinaciones posibles de parámetros o variables, pero parte siempre de unos datos bien preparados. Por tanto, es importante aplicar las transformaciones necesarias al principio del proceso, tal como se ha hecho en otras unidades.

Variables de un algoritmo genético

Un algoritmo genético trabaja con una población de soluciones posibles que evolucionan con el tiempo. Cada una de estas soluciones se representa mediante un cromosoma, que contiene los genes, es decir, las piezas de información que definen la solución.

Estas son las variables más importantes que siempre están presentes en cualquier algoritmo genético, independientemente del problema que resuelva:

Principales variables del sistema:

- **Cromosoma:** Es una representación de una solución. Puede ser una lista de números, caracteres, valores binarios, etc. Cada cromosoma está compuesto por varios genes.
- **Gen:** Es la unidad mínima del cromosoma. Representa una parte concreta de la solución. Por ejemplo, una letra en una contraseña o una ciudad en una ruta.
- **Población:** Conjunto de cromosomas que se evalúan en cada generación. Es decir, todas las soluciones candidatas que compiten y evolucionan en cada ciclo.
- **Tamaño de la población:** Número de individuos que componen la población. Afecta al equilibrio entre diversidad y velocidad del algoritmo.
- **Máximo de generaciones:** Número de ciclos evolutivos que se van a ejecutar. Se puede usar como condición de parada.
- **Función objetivo:** Define qué solución es ideal o qué se busca alcanzar. Puede ser explícita (una contraseña exacta) o implícita (minimizar una distancia).

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



- **Función de aptitud (fitness):** Evalúa qué tan buena es una solución. A mayor fitness, más posibilidades de que ese individuo se reproduzca.

Ejemplo típico: descubrimiento de contraseñas

Uno de los ejemplos clásicos de algoritmos genéticos es intentar descubrir una contraseña desconocida. En este caso:

- Cada **cromosoma** sería una cadena de caracteres del mismo tamaño que la contraseña objetivo.
- Cada **gen** sería un carácter individual (una letra, un número o un símbolo).
- La **población** estaría formada por múltiples cadenas generadas aleatoriamente.
- La **función de fitness** compararía cada cromosoma con la contraseña real y daría más puntuación cuanto más se parezca.

Fases de un algoritmo genético

Un algoritmo genético funciona por ciclos. En cada generación, las soluciones se evalúan, se reproducen y se modifican con el objetivo de mejorar la calidad de la población. Estas son las fases principales que se repiten en cada iteración:

1. **Inicialización:** Se crea una población inicial de cromosomas aleatorios. Cada uno representa una solución posible al problema. Esta población debe ser lo bastante diversa como para explorar distintas zonas del espacio de soluciones.
2. **Evaluación (fitness):** Se evalúa la calidad de cada individuo mediante una función de aptitud o *fitness*. Esta función asigna una puntuación a cada cromosoma según lo cerca que esté de la solución ideal. Cuanto mejor sea su rendimiento, más probabilidad tendrá de ser seleccionado.
3. **Selección:** Se eligen los individuos que van a reproducirse. La probabilidad de ser seleccionado suele estar relacionada con su fitness: los mejores tienen más oportunidades. Hay distintos métodos de selección:
 - a. **Selección por ruleta (roulette wheel):** Cada individuo tiene una probabilidad de ser elegido proporcional a su valor de fitness. Es como una ruleta en la que los individuos más aptos ocupan más espacio y, por tanto, tienen más probabilidades de ser seleccionados. Es sencilla de implementar y permite mantener cierta diversidad.
 - b. **Selección por torneo:** Se eligen al azar dos o más individuos de la población y gana el que tenga mayor fitness. Este proceso se repite para formar todas las parejas. Permite controlar la presión selectiva ajustando el tamaño del torneo: si es mayor, hay más probabilidad de que ganen los mejores.
 - c. **Elitismo:** No es un método de selección en sí, sino una estrategia complementaria. Consiste en guardar directamente los mejores individuos de una generación y pasarlos tal cual a la siguiente. Así se evita perder buenas soluciones por azar.
 - d. **Selección por ranking:** En lugar de usar directamente el valor de fitness, se ordenan los individuos de mejor a peor y se asignan probabilidades según su posición. Esto evita que un individuo muy bueno domine completamente si su fitness está muy por encima del resto.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



4. **Emparejamiento y cruce (crossover):** Se forman parejas de individuos seleccionados y se mezclan sus genes para generar nuevas soluciones. El cruce puede hacerse de muchas formas: intercambiando partes del cromosoma, combinando valores, etc. El objetivo es aprovechar lo mejor de cada “progenitor”.
5. **Mutación:** Algunos genes de los nuevos cromosomas se modifican aleatoriamente. Esto introduce diversidad genética y evita que el algoritmo se quede estancado en soluciones mediocres. La mutación suele aplicarse con baja probabilidad.
6. **Reemplazo:** La nueva generación sustituye parcial o totalmente a la anterior. A veces se mantiene el mejor individuo (elitismo) para asegurar que no se pierda la mejor solución hallada hasta el momento.

Este ciclo se repite durante varias generaciones. El proceso termina cuando se alcanza el número máximo de generaciones, o cuando se encuentra una solución suficientemente buena según el criterio del problema.

Ventajas frente a métodos clásicos

Los algoritmos genéticos, como cualquier metaheurística, no forman parte del aprendizaje automático, ya que no aprenden a partir de datos ni construyen modelos predictivos. Sin embargo, sí pueden aplicarse a los mismos problemas que aparecen en machine learning, especialmente cuando hay que optimizar combinaciones de parámetros, seleccionar variables o ajustar configuraciones complejas.

En muchos de estos casos, los genéticos resultan más eficaces que los métodos clásicos porque:

- **Exploran mejor el espacio de soluciones:** No se limitan a probar combinaciones fijas (como grid search) ni a elegir las al azar (como random search), sino que adaptan la búsqueda generación tras generación, guiándose por la calidad de las soluciones anteriores.
- **No necesitan información matemática del problema:** Funcionan incluso si la función a optimizar no tiene derivadas, es discontinua o no es suave. No necesitan saber cómo es la función, sólo evaluar si una solución es mejor o peor.
- **Permiten escapar de óptimos locales:** Gracias a la mutación y al cruce, son capaces de saltar a otras zonas del espacio de búsqueda, mientras que métodos como el descenso por gradiente pueden quedar atrapados en una solución parcial.
- **Se adaptan a cualquier tipo de variable:** Pueden trabajar con enteros, reales, cadenas, vectores binarios o combinaciones de todo. No requieren adaptar el algoritmo a cada caso.
- **Escalables y paralelizables:** Al evaluar muchos individuos en paralelo, se pueden distribuir fácilmente en sistemas con varios núcleos o GPUs.

Los algoritmos genéticos son una herramienta poderosa para resolver problemas de optimización complejos, especialmente cuando el espacio de soluciones es muy grande o no estructurado. Aunque no forman parte del núcleo del machine learning, se adaptan muy bien a tareas relacionadas, como la búsqueda de hiperparámetros o la selección de variables. Gracias a su flexibilidad,

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



simplicidad conceptual y capacidad para escapar de óptimos locales, siguen siendo una alternativa eficaz y práctica frente a métodos clásicos.

Cierre de la unidad

En esta unidad hemos recorrido distintas estrategias que permiten mejorar el rendimiento de los modelos de aprendizaje automático. Hemos trabajado con redes neuronales más allá del perceptrón multicapa, hemos aprovechado el potencial de modelos preentrenados para reutilizar conocimiento y hemos descubierto cómo técnicas inspiradas en la naturaleza, como los algoritmos genéticos, pueden servir para optimizar decisiones complejas.

A lo largo del temario hemos visto que no hay un único camino correcto. A veces una red neuronal simple bien entrenada da mejores resultados que un modelo complejo. Otras veces, ajustar bien los hiperparámetros o seleccionar correctamente las variables tiene más impacto que cambiar de algoritmo. Por eso, uno de los aprendizajes clave es saber cuándo conviene aplicar una técnica, cuándo es suficiente lo básico y cuándo merece la pena explorar herramientas más avanzadas.

Todo lo que hemos visto forma parte de un campo que está en constante evolución. Existen muchas más arquitecturas, técnicas de optimización, enfoques híbridos y formas de adaptar los modelos a distintos problemas. Este recorrido no pretende cubrirlo todo, sino dar una base sólida para entender cómo funcionan las decisiones que se toman al entrenar un modelo y abrir la puerta a seguir aprendiendo con criterio.

Más allá del modelo, el aprendizaje automático es una combinación de conocimiento, exploración y sentido práctico.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)

